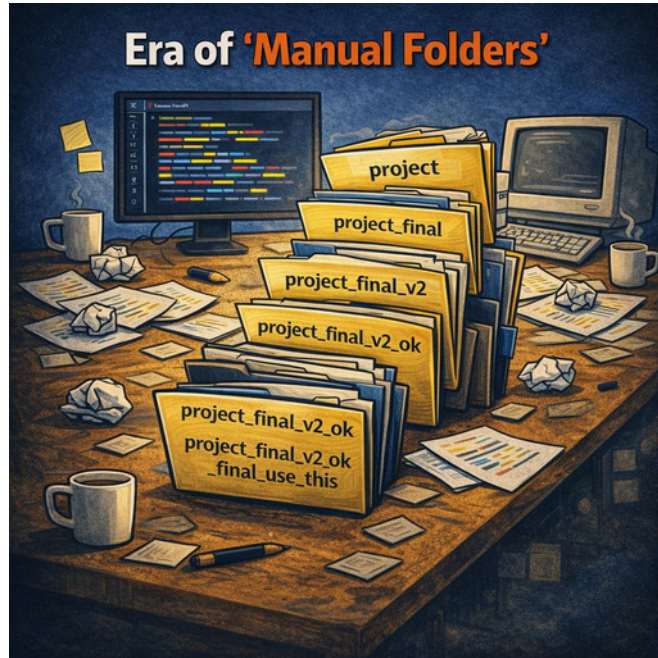


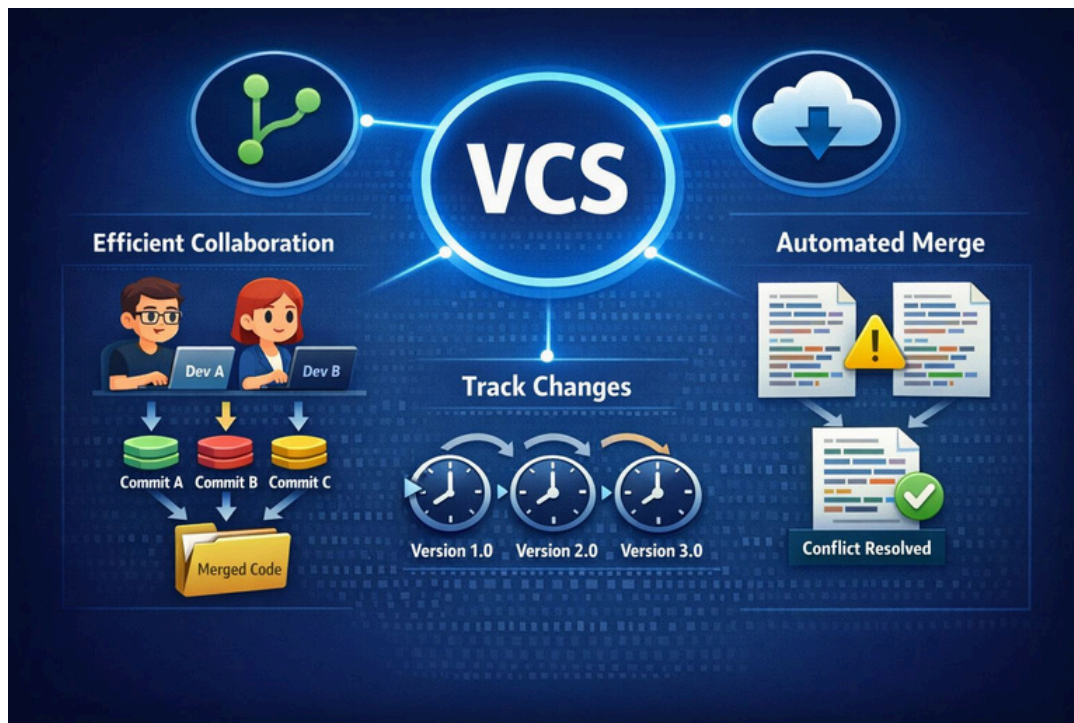
# Git vs Github

## NỘI DUNG

1. Tổng quan về VCS, Git & GitHub
2. Các lệnh Git căn bản thường dùng
3. Git Workflow trong các dự án thực tế
4. Các lệnh Git thông dụng trong dự án
5. Thực hành Case Study, hỏi đáp (Q&A)



Merge Conflict thủ công những năm 1990 - đầu 2000



VCS ra đời xử lý vấn đề Merge Conflict file thủ công

- **Đặc điểm của VCS (Version Control System)**

1. Lưu trữ và quản lý lịch sử thay đổi code Quay lại phiên bản cũ (**Rollback**)
2. Làm việc nhóm hiệu quả Quản lý nhiều phiên bản song song (**Branching**)
4.
  - a. Tạo nhánh feature
  - b. Fix bug riêng biệt
  - c. Tách môi trường (**main, dev, feature,...**)
5. Phát hiện và xử lý xung đột (**Conflict**)
6. Code review & kiểm soát chất lượng
7. Đồng bộ code giữa các môi trường
8. Nền tảng cho CI/CD & DevOps

- **Có 2 loại VCS:** Centralized VCS, Distributed VCS

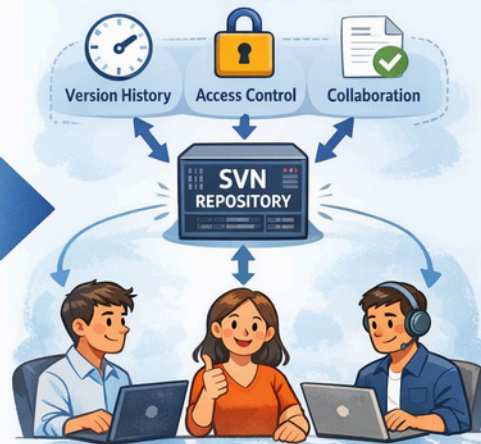


## SVN Ra Đời Giúp Cải Thiện Quản Lý File

### Quản Lý Thủ Công



### Quản Lý Với SVN



**SVN (Centralized VCS)** ra đời, giải quyết các nỗi đau của việc Quản lý file, Merge Conflict file thủ công



## ƯU ĐIỂM CỦA SVN

### Mô hình tập trung



Quản lý tập trung

### Dễ học, dễ sử dụng



Thao tác đơn giản

### Quản lý theo Revision



Theo dõi phiên bản

### Khóa file



Bảo vệ file binary

### Không cần clone toàn bộ



Tiết kiệm dung lượng

### Phân quyền chi tiết



Giới hạn truy cập

## Nhược Điểm của SVN



Không hỗ trợ làm việc Offline



Xung đột đồng bộ



Tốc độ chậm



Quyền truy cập hạn chế

Ưu, nhược điểm của SVN



**Git (Centralized VCS)** ra đời, khắc phục hết nhược điểm của SVN. **Git** - Hệ thống quản lý phiên bản mã nguồn dạng phân tán

# 1. Tổng quan về VCS, Git & GitHub



**GitHub** là một dịch vụ **Git** được cung cấp miễn phí (Và cũng có phiên bản trả phí cho doanh nghiệp)



# 1. Tổng quan về VCS, Git & GitHub

- Sự khác nhau giữa Git & GitHub

| Tiêu chí        | Git                      | GitHub                          |
|-----------------|--------------------------|---------------------------------|
| Bản chất        | Công cụ quản lý mã nguồn | Nền tảng lưu trữ mã nguồn       |
| Chạy ở đâu      | Local (máy cá nhân)      | Cloud (internet)                |
| Chức năng chính | Quản lý version code     | Lưu trữ + cộng tác nhóm         |
| Có cần internet | Không                    | Có                              |
| Làm việc nhóm   | Hạn chế (local)          | Rất mạnh                        |
| Ví dụ lệnh      | git commit, git branch   | Không có lệnh (chỉ có UI / API) |

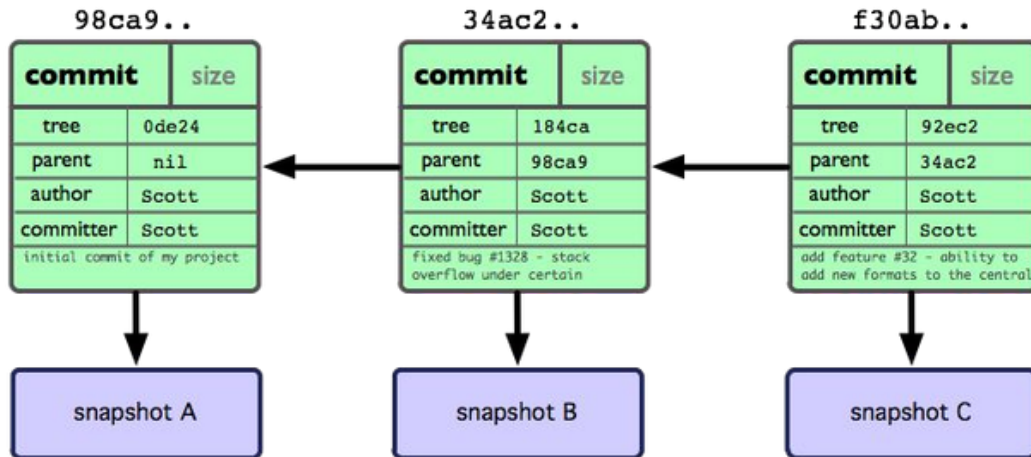
# 1. Tổng quan về VCS, Git & GitHub

- **Snapshot:**
  - Toàn bộ mã nguồn tại một thời điểm Lập trình viên
  - quyết định lúc nào tạo snapshot Có thể quy lại một
  - snapshot bất kỳ
- **Commit:**
  - Cách để tạo ra các snapshot
  - Thường được tạo khi có một thay đổi đáng kể:
    - Tạo một tính năng mới
    - Sửa một lỗi Cải tiến mã
    - nguồn



# 1. Tổng quan về VCS, Git & GitHub

- Một commit bao gồm các thông tin:
  - Thay đổi so với file trước Một tham chiếu đến commit trước đó Một mã băm đại diện, thường có
  - dạng: **87878747939740429190ca307289c494311e27fe**
  -

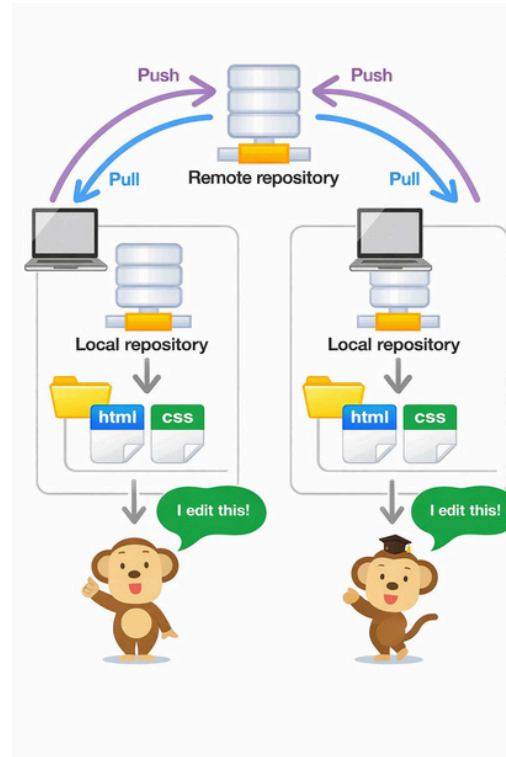


# 1. Tổng quan về VCS, Git & GitHub

- **Repository:**

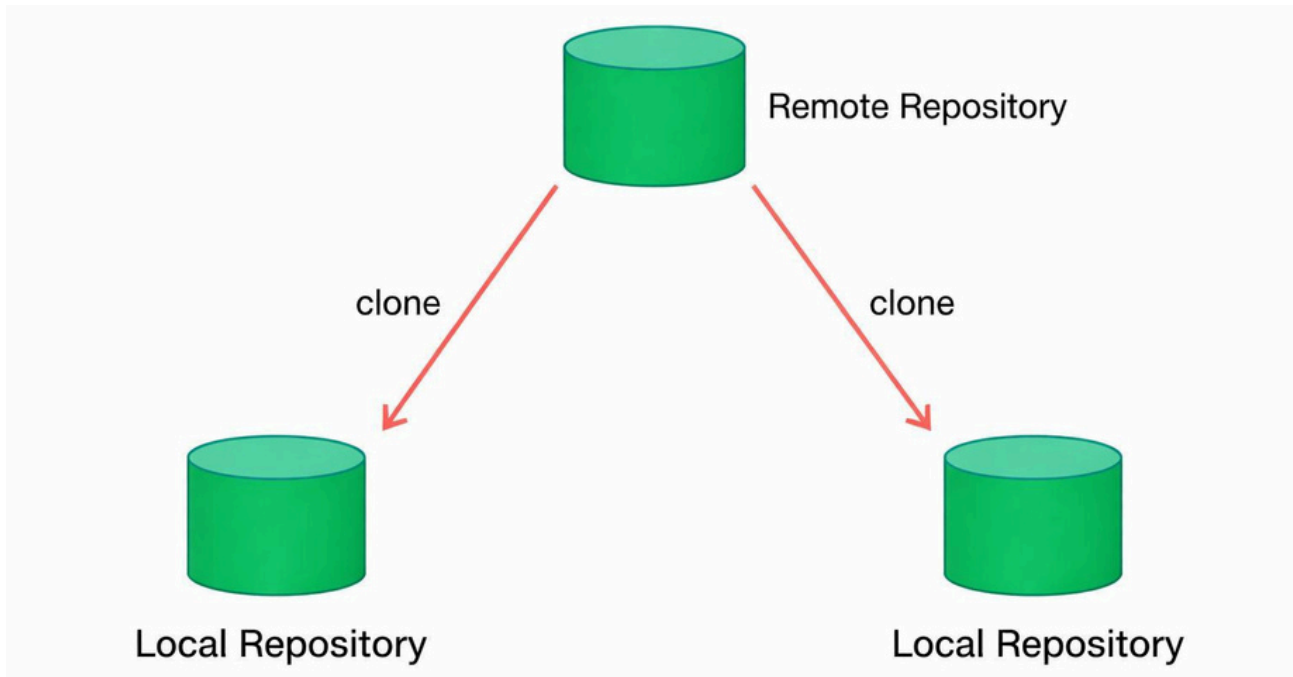
- Thường được gọi ngắn gọn là **repo** Nơi chứa toàn bộ
- mã nguồn Bao gồm toàn bộ các file, lịch sử của các
- file đó Chứa tất cả các commit của dự án Có 2 loại:

- **Local Repository:** Kho chứa code ở trên máy tính của lập trình viên
- **Remote Repository:** Kho chứa code ở trên máy chủ chia sẻ (ví dụ: GitHub)



# 1. Tổng quan về VCS, Git & GitHub

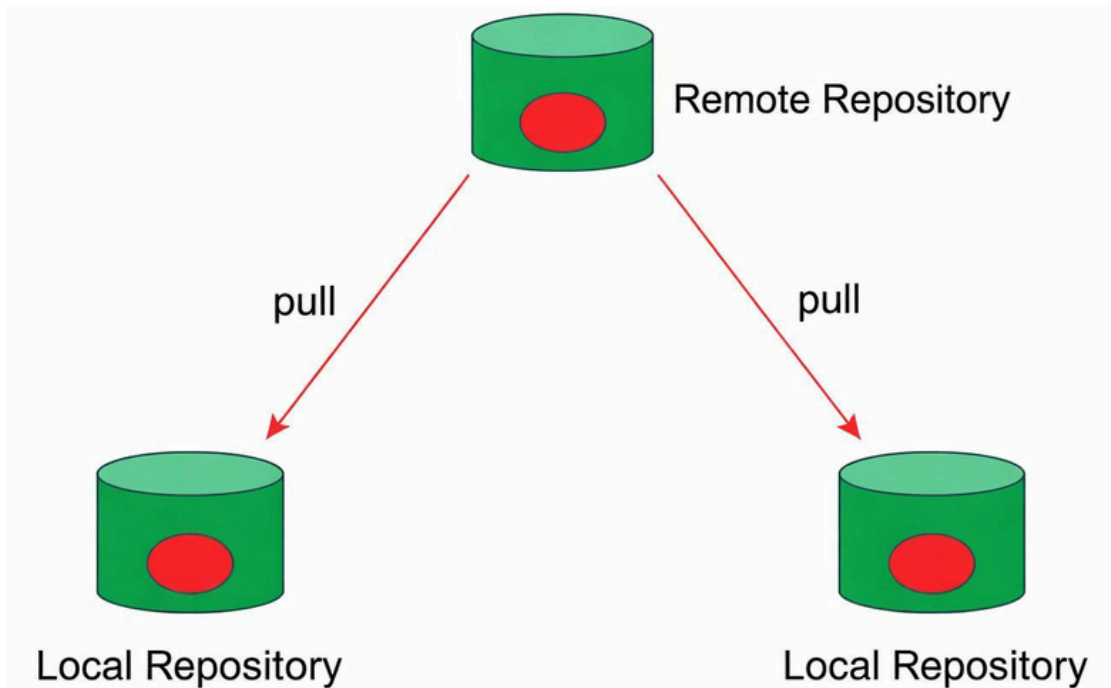
- **Clone:** sao chép một **Remote Repository** về làm **Local Repository** ở máy của lập trình viên





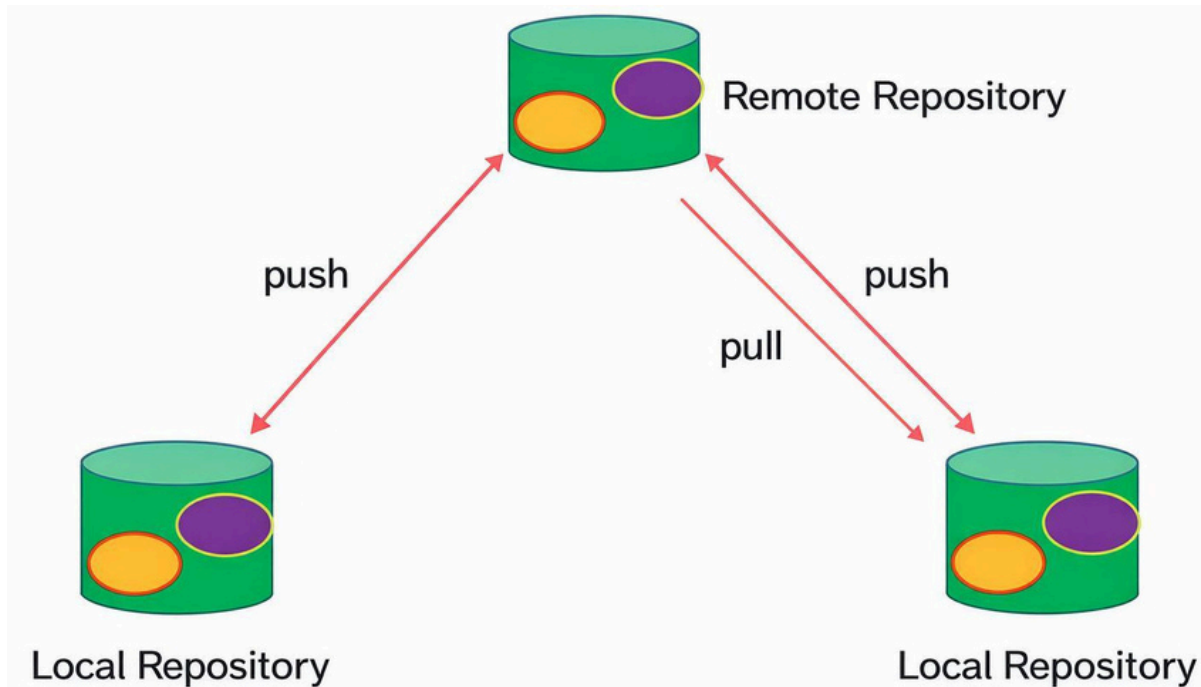
# 1. Tổng quan về VCS, Git & GitHub

- **Pull:** cập nhật mã nguồn từ một **Remote Repository** về **Local Repository**

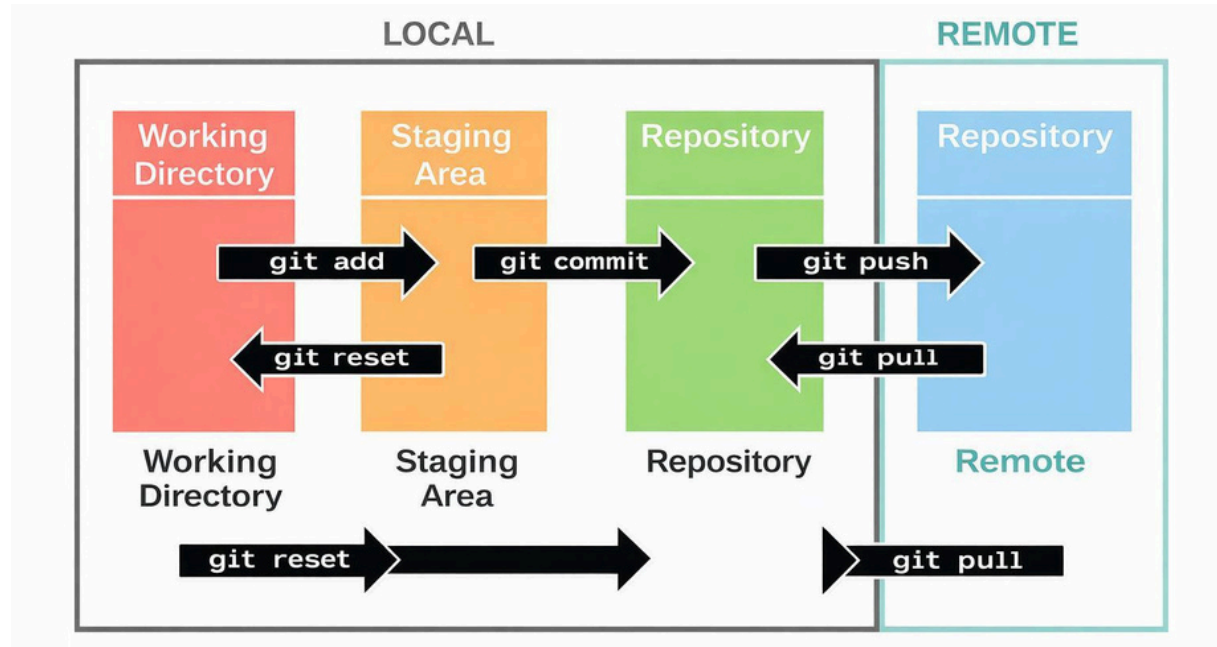


# 1. Tổng quan về VCS, Git & GitHub

- **Push:** đẩy mã nguồn từ một **Local Repository** lên **Remote Repository**



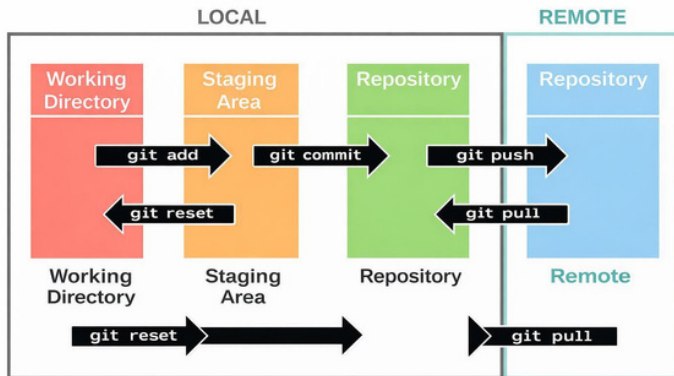
# 1. Tổng quan về VCS, Git & GitHub



Các khu vực làm việc của Git

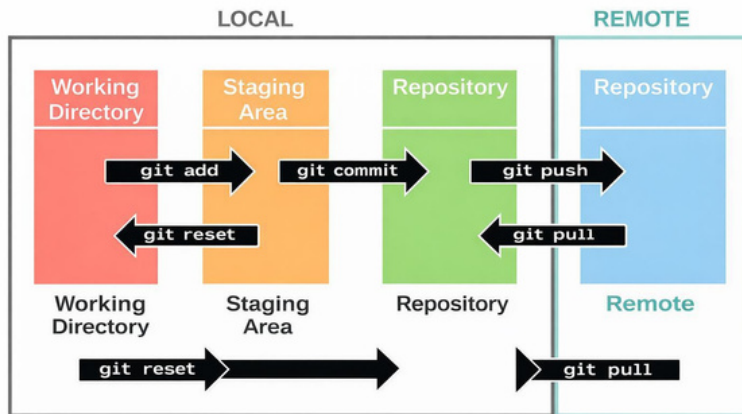
## 2. Các lệnh Git căn bản thường dùng

- Các lệnh Git căn bản (khởi tạo):
  - **git clone**: sao chép một **Remote Repository** về thành **Local Repository**
  - **git init**: khởi tạo một **Local Repository** trên máy tính của lập trình viên
  - **git config**: thiết lập thông tin người dùng **Git** để ghi vào mỗi **commit**
  - **git remote**: quản lý địa chỉ các kho remote như **GitHub**



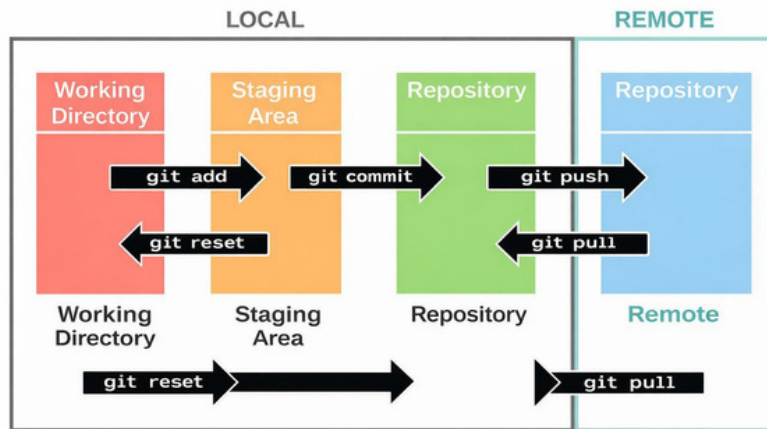
## 2. Các lệnh Git căn bản thường dùng

- Các lệnh Git căn bản (lưu code):
  - **git add**: đưa các file thay đổi vào trong vực lưu vết thay đổi (**Staging Area**) **git commit**: ghi nhận
  - các thay đổi từ **Staging Area** vào **Local Repository** **Lưu ý**: nên đặt tên commit tường minh mô tả
  - được những thay đổi của mã nguồn



## 2. Các lệnh Git căn bản thường dùng

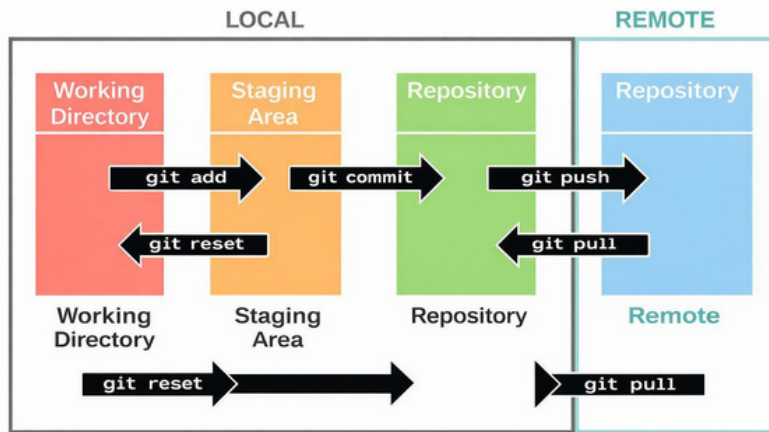
- Các lệnh Git căn bản (kiểm tra):
  - **git status**: sao chép một **Remote Repository** về thành **Local Repository**
  - **git log**: khởi tạo một **Local Repository** trên máy tính của lập trình viên
  - **git diff**: sử dụng để so sánh thay đổi trong code khi chưa **commit**





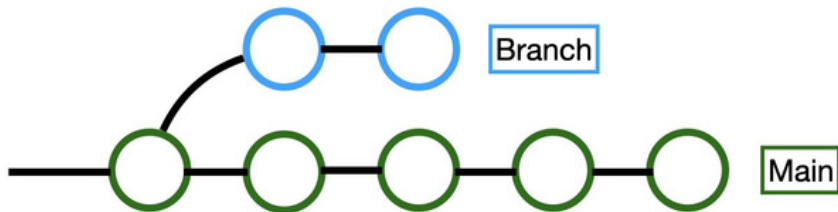
## 2. Các lệnh Git căn bản thường dùng

- Các lệnh Git căn bản (làm việc nhóm):
  - **git pull**: lấy thay đổi mới từ **Remote Repository** về và gộp luôn vào **Local Repository**
  - **git push**: đẩy **commit** từ **Local Repository** lên **Remote Repository** **Lưu ý**: Nên kiểm tra kỹ trước khi push, tránh push trực tiếp lên nhánh mặc định (main / dev)



## 2. Các lệnh Git căn bản thường dùng

- **Nhánh (branch):**
  - Là một chuỗi các commit. Tạo nhánh sẽ tạo không gian an toàn để làm việc mà không ảnh hưởng đến nhánh chính
  - nhánh chính
- **Quản lý nhánh & tần suất tích hợp**
  - **Mainline** (nhánh chính): **master** hoặc **main**, đại diện cho trạng thái hiện tại của sản phẩm
  - **Tần suất tích hợp**: Tích hợp thường xuyên giúp các bản merge nhỏ hơn, ít rủi ro hơn và phát hiện xung đột sớm hơn.



### 3. Git Workflow trong các dự án thực tế

- So sánh việc sử dụng Git trong trường học và trong thực tế doanh nghiệp

| Tiêu chí       | Git trong trường học          | Git trong thực tế                   |
|----------------|-------------------------------|-------------------------------------|
| Mục tiêu       | Nộp bài, quản lý code cá nhân | Làm việc nhóm, quản lý dự án        |
| Số người       | Cá nhân hoặc nhóm nhỏ         | Team nhiều người (5-20+)            |
| Sử dụng branch | Ít hoặc không dùng            | Bắt buộc dùng (feature, bugfix,...) |
| Workflow       | Đơn giản (commit -> push)     | Có quy trình rõ (PR, review, CI/CD) |
| Conflict       | Ít gặp                        | Gặp thường xuyên                    |
| Hiểu Git       | Biết lệnh cơ bản              | Hiểu workflow + chiến lược          |
| Deploy         | Không hoặc thủ công           | Tự động (CI/CD pipeline)            |

### 3. Git Workflow trong các dự án thực tế

- **Có 3 Git Workflow phổ biến:** Git Flow, GitHub Flow, Trunk-Based Development (TBD)

| Tiêu chí        | Git Flow                          | GitHub Flow            | Trunk-Based Development                   |
|-----------------|-----------------------------------|------------------------|-------------------------------------------|
| Mục tiêu        | Quản lý nhiều version             | Đơn giản, deploy nhanh | Tối ưu CI/CD, DevOps                      |
| Phù hợp         | Sản phẩm đóng gói (multi-version) | Web App, SaaS          | Hệ thống lớn, Agile                       |
| Số lượng branch | Nhiều                             | Ít                     | Rất ít                                    |
| Nhánh chính     | master (prod), develop (dev)      | main                   | Main (trunk)                              |
| Nhánh phụ       | feature, release, hotfix          | feature branch         | feature branch cực ngắn (hoặc không dùng) |

### 3. Git Workflow trong các dự án thực tế

- **Có 3 Git Workflow phổ biến:** Git Flow, GitHub Flow, Trunk-Based Development (TBD)

| Tiêu chí           | Git Flow                   | GitHub Flow       | Trunk-Based Development      |
|--------------------|----------------------------|-------------------|------------------------------|
| Quy trình làm việc | Phức tạp, nhiều bước       | Đơn giản, dễ hiểu | Liên tục, tốc độ cao         |
| Deploy             | Theo version release       | Deploy liên tục   | Deploy liên tục (CI/CD mạnh) |
| Kiểm soát code     | Qua nhiều nhánh trung gian | Qua Pull Request  | Qua CI + test tự động        |
| Tốc độ phát triển  | Trung bình                 | Nhanh             | Rất nhanh                    |
| Rủi ro conflict    | Cao (do nhiều branch dài)  | Trung bình        | Thấp (merge liên tục)        |

### 3. Git Workflow trong các dự án thực tế

- **Có 3 Git Workflow phổ biến:** Git Flow, GitHub Flow, Trunk-Based Development (TBD)

| Tiêu chí        | Git Flow           | GitHub Flow      | Trunk-Based Development              |
|-----------------|--------------------|------------------|--------------------------------------|
| Độ khó          | Khó                | Dễ               | Trung bình - khó (cần kỷ luật cao)   |
| Kỹ thuật đi kèm | Release management | Code Review (PR) | Feature Flags, Branch by Abstraction |

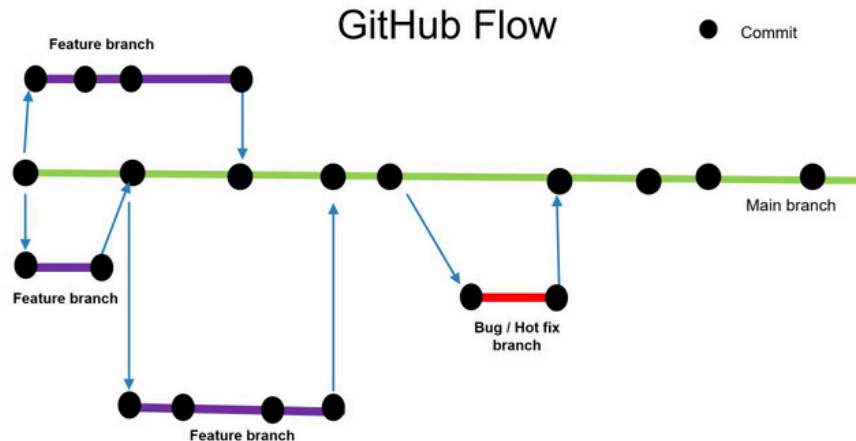
- **Các loại dự án phù hợp:**
  - **Git Flow** -> nhiều branch -> quản lý version -> **phù hợp sản phẩm lớn, nhiều khách hàng**
  - **GitHub Flow** -> đơn giản -> PR -> merge -> **phù hợp web/app hiện đại**
  - **Trunk-Based** -> commit liên tục vào main -> **phù hợp với dự án DevOps, CI/CD mạnh**

=> Trong thực tế hiện nay (từ 2020 trở đi), **GitHub Flow là workflow Git phổ biến nhất**

### 3. Git Workflow trong các dự án thực tế

- Giới thiệu GitHub Flow:

- Quy trình làm việc dựa trên các nhánh (**branch-based workflow**) rất tinh gọn, nhẹ nhàng Được thiết kế đặc biệt
- cho các hệ thống duy trì một phiên bản **production** duy nhất Ưu tiên tích hợp liên
- tục, giữ nhánh chính (**master** hoặc **main**) luôn trong trạng thái sẵn sàng **release**





### 3. Git Workflow trong các dự án thực tế

- **Cách đặt tên nhánh theo GitHub Flow:**

- ❑ Ngắn gọn và mang tính mô tả trực diện -> Ví dụ: **implement-login-feature**
- ❑ Tách biệt cho từng cụm công việc -> Ví dụ Đặt **KHÔNG NÊN**: **update-login-and-fix-payment**
- ❑ tên theo môi trường (Environment)
  - **production / prod**: môi trường thực tế
  - **staging / stag**: môi trường trước khi phát hành
  - **test / qa**: môi trường kiểm thử chất lượng
  - **develop / dev**: môi trường phát triển
  -

### 3. Git Workflow trong các dự án thực tế

- Cách đặt tên nhánh theo GitHub Flow:



Sử dụng tiền tố phân loại ( **Prefixes**)

- **release/** : Dành cho nhánh ổn định mã nguồn trước khi phát hành lên **production**
- **hotfix/** : Dành cho nhánh sửa lỗi khẩn cấp trên **production**
- **feat/** hoặc **feature/** : Dành cho phát triển tính năng mới
- **fix/** hoặc **bugfix/** : Dành cho việc sửa lỗi
- **exp/** hoặc **/experiment/** : Dành cho các thử nghiệm, không chắc chắn sẽ merge
- **Lưu ý:** trong **GitHub Flow** có thể có hoặc không cần 2 loại nhánh **hotfix/** và **release/**
-

### 3. Git Workflow trong các dự án thực tế

- Cách đặt tên nhánh theo GitHub Flow:



Gắn kèm mã số quản lý dự án ( **Ticket/Issue ID**)

- **Pull Request** là cơ chế để **review code** -> cần liên kết PR với các issue để theo dõi
- Các cty thường đưa trực tiếp mã số quản lý dự án (**Jira, Trello,...**) vào tên nhánh
- Công thức đặt tên: **[tiền\_tô]/[Mã\_số\_issue]-[Mô\_tả\_ngắn\_gọn]**

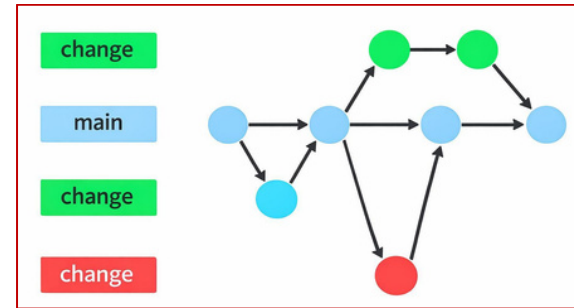
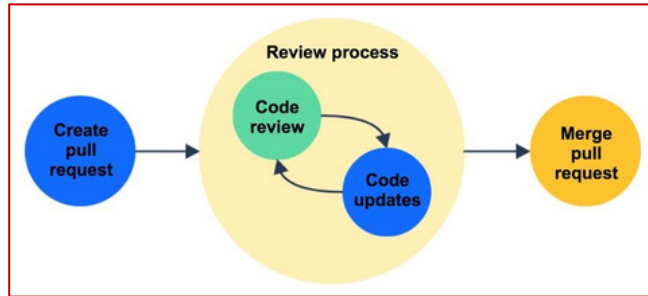
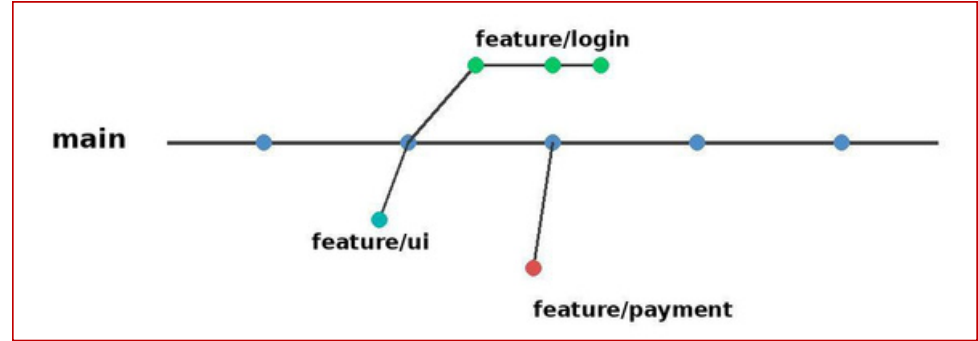
Ví dụ:

- - **feat/JIRA-123-implement-login-feature**
  - hoặc **fix/ISSUE-456-fix-login-crash**

### 3. Git Workflow trong các dự án thực tế

- Quy trình GitHub Flow:

- Bước 1:** Tạo nhánh
- Bước 2:** Thực hiện thay đổi
- Bước 3:** Tạo Pull Request (PR)
- Bước 4:** Xử lý phản hồi
- Bước 5:** Hợp nhất PR
- Bước 6:** Xóa nhánh



## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git** restore):
  - **Công dụng:** hủy tất cả các thay đổi chưa commit trong **Local Repository** Cách sử dụng:
    - Hủy thay đổi 1 file: **git restore file.java**
    - Hủy thay đổi nhiều file: **git restore .**
    - Bỏ file khỏi staging: **git restore --staged file.java** (**quay về commit cũ**)
    - Khôi phục từ commit cũ: **git restore --source=<commit-id> file.java**
    -

## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git** stash):
  - **Công dụng:** lưu trữ tạm thời các thay đổi chưa commit **Local Repository\ Cách sử dụng:**
  - **Bước 1:** Lưu code đang làm dở ở nhánh **feature-01** bằng lệnh: **git stash**
  - **Bước 2:** Xem danh sách các thay đổi bằng lệnh: **git stash list**
  - **Bước 3:** Lấy lại code đã stash bằng lệnh:
    - **Cách 1:** Lấy và xóa luôn stash trong stash list: **git stash pop** (dùng nhiều nhất)
    - **Cách 2:** Lấy nhưng giữ lại stash trong stash list: **git stash apply**

## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git** stash):
  - Trường hợp thực tế sử dụng **git** stash:

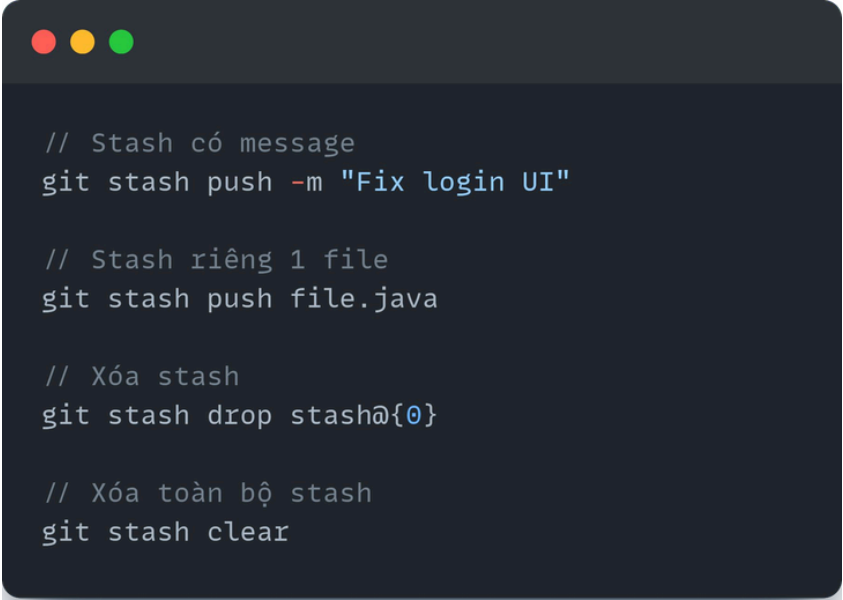
```
// Case 1: Đang code dở → cần chuyển nhánh
git stash
git switch main

// Case 2: Pull code mới về
git stash
git pull origin main
git stash pop

// Case 3: Fix bug gấp
git stash
git switch hotfix/bug
```

## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git** stash):
  - Các lệnh **git stash** nâng cao nên biết:

A dark-themed terminal window with three colored window control buttons (red, yellow, green) at the top left. It contains four lines of Git commands, each preceded by a comment in Vietnamese. The commands are: 'git stash push -m "Fix login UI"', 'git stash push file.java', 'git stash drop stash@{0}', and 'git stash clear'.

```
// Stash có message
git stash push -m "Fix login UI"

// Stash riêng 1 file
git stash push file.java

// Xóa stash
git stash drop stash@{0}

// Xóa toàn bộ stash
git stash clear
```



## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git revert** và **git reset**):
  - **git revert**: tạo một commit mới để đảo ngược **commit cũ đã push lên remote** **git reset**: di chuyển **HEAD** về **commit cũ trước đó chưa push lên remote**

| Tiêu chí                | git revert      | git reset          |
|-------------------------|-----------------|--------------------|
| Mục tiêu Cách hoạt động | Hoàn tác commit | Quay ngược lịch sử |
| Mất lịch sử             | Tạo commit mới  | Di chuyển HEAD     |
| không? An toàn          | Không           | Có thể             |
| teamwork                | An toàn         | Không an toàn      |
| Sử dụng khi             | <b>Đã push</b>  | <b>Chưa push</b>   |

## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git** revert):
  - Cú pháp: **git** revert <commit-id> Cách
  - hoạt động:

A --- B --- C (bad commit)

revert -> A --- B --- C --- D

↑

D = undo C

=> Không xóa commit C, chỉ tạo commit D để đảo ngược code

- **Lưu ý:** chỉ sử dụng khi code **ĐÃ PUSH** lên GitHub

## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git** reset):
  - Cú pháp: **git** reset <Mức\_độ\_reset\_commit> **HEAD~<quay\_lại\_n\_commit>** Các mức độ
  - reset commit:
    - **--soft**: **git** reset **--soft HEAD~1** -> Xóa 1 commit trước đó chưa push, giữ code
    - **--default**: **git** reset **HEAD~1** -> Xóa 1 commit trước đó chưa push, bỏ staging
    - **--hard**: **git** reset **--hard HEAD~1** -> Xóa 1 commit trước đó chưa push, mất code
    -

## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git** reset):

- Cách hoạt động:

A --- B --- C (bad commit) reset -> A --- B =>

commit C có thể bị xóa khỏi lịch sử

- Lưu ý:

- Mất lịch sử, mất code, gây lỗi khi làm việc nhóm

- Chỉ sử dụng khi code **CHƯA PUSH** lên GitHub

## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git fetch**, **git merge**, **git pull**):

| Lệnh             | Làm gì                | Có merge không? |
|------------------|-----------------------|-----------------|
| <b>git fetch</b> | Lấy code từ remote về | Không Có Có     |
| <b>git merge</b> | Gộp branch            |                 |
| <b>git pull</b>  | Fetch + Merge         |                 |

- Cách sử dụng thực tế (an toàn): **git fetch origin**  
**git merge origin/main**
- Cách sử dụng thực tế (nhanh, gọn): **git pull origin main**

## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git merge**, **git rebase**):
  - **git merge**: dùng để **gộp code** từ một **branch khác** vào **branch hiện tại**
  - **git rebase**: di **chuyển các commit** của **branch hiện tại** lên **một base mới**

| Tiêu chí          | <b>git merge</b> | <b>git rebase</b>        |
|-------------------|------------------|--------------------------|
| Bản chất          | Gộp 2 nhánh      | Dời commit sang base mới |
| Lịch sử commit    | Giữ nguyên       | Viết lại                 |
| Tạo merge commit? | Có               | Không                    |
| Độ an toàn        | Cao              | Cẩn thận khi dùng        |
| Dễ hiểu           | Dễ hơn           | Khó hơn                  |

## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git** merge):
  - Cú pháp: **git** checkout <nhánh\_đích\_nơi\_sẽ\_gộp\_code\_nhánh\_khác\_vào> **git** merge <nhánh\_chứa\_code\_muốn\_gộp\_vào\_nhánh\_hiện\_tại>

- Cách hoạt động:

main:      A --- B --- C

                 \

feature:                      D --- E

merge ->

main:      A --- B --- C ----- F

                 \

                 /

                 D --- E

## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git** rebase):
  - Cú pháp: **git** checkout <branch\_01>  
**git** rebase <branch\_02>
  - Trong đó:
    - **branch\_01**: Nhánh chứa các commit cần di chuyển sang nhánh khác **branch\_02**: Nhánh đích nơi sẽ chứa các commit mà nhánh hiện tại cần dời đi



## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git** rebase):

- **Trạng thái ban đầu:**

main: A --- B --- C

                  \   D -

feature:           -- E

- **Team update thêm commit mới lên **main**:**

main:       A --- B --- C --- F --- G

                  \

feature:           D --- E

## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git rebase**):

- Vấn đề gặp phải -> **feature** đang lệch so với **main**:

- Nếu **git merge** -> lịch sử rối Nếu

- **git rebase** -> lịch sử sạch

- Thực hiện rebase để di chuyển commit từ **feature** lên **main**: **git**

**checkout feature/login**

**git rebase main**

- Kết quả sau khi rebase:

main:        A --- B --- C --- F --- G --- D' --- E'

- **Lưu ý bị conflict khi rebase**: giải quyết conflict -> tiếp tục rebase: **git rebase --continue**

## 4. Các lệnh Git thông dụng trong dự án

- Các lệnh Git thông dụng (**git cherry-pick**):
  - **git cherry-pick**: dùng để lấy 1 (hoặc vài) commit từ branch khác sang branch hiện tại **Cú pháp: git**
  - **cherry-pick <commit-id>** (**cherry-pick** là **COPY commit**, không dời commit) Trước khi dùng **git cherry-**
  - **pick**:

main:        A --- B --- C

             \

feature:        D --- E --- F

- Sau khi dùng **git cherry-pick** -> **E' # E** (commit ID của E, E' sẽ khác nhau):

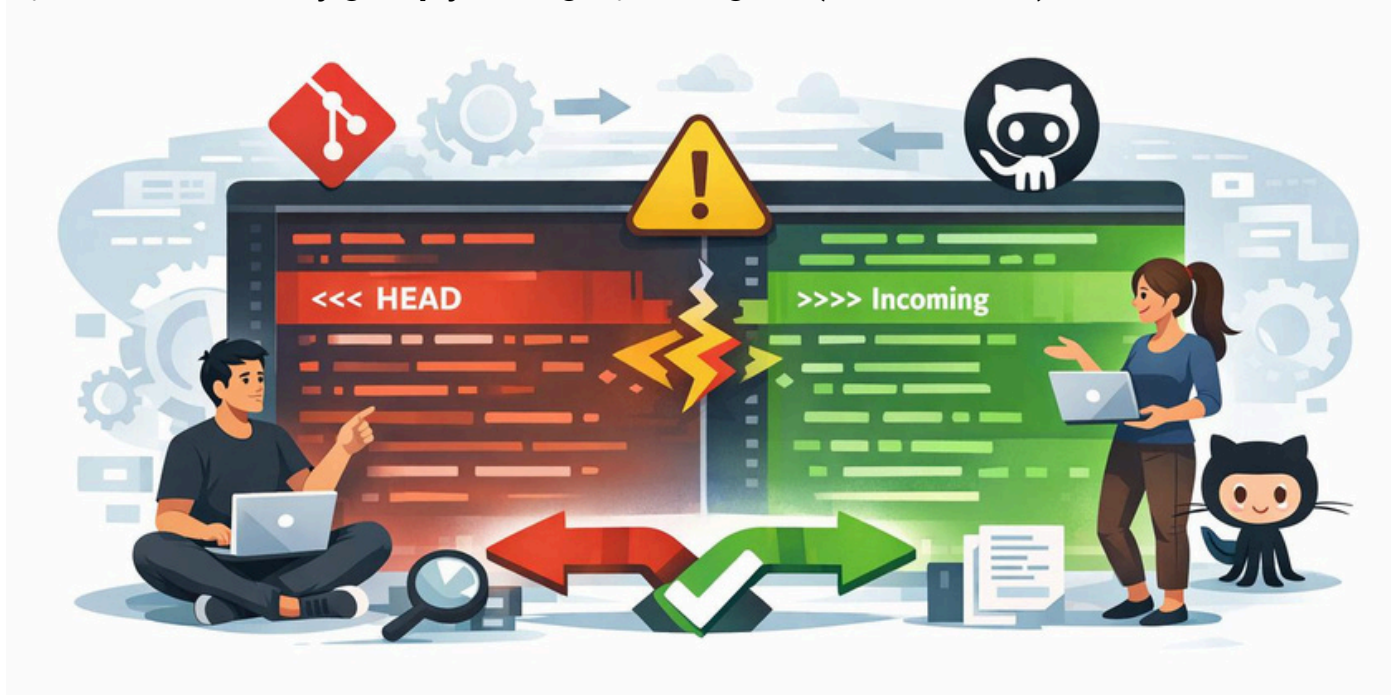
main:        A --- B --- C --- E'

             \

feature:        D --- E --- F

## 5. Thực hành Case Study, hỏi đáp (Q&A)

- Thực hành Case Study giải quyết xung đột mã nguồn (Conflict Code) với Git, GitHub



# TỔNG KẾT

- VCS (Version Control System) là hệ thống quản lý phiên bản mã nguồn ra đời nhằm khắc phục nhược điểm của quản lý file thủ công
- VCS có 2 loại chính đó là Centralized VCS (SVN) và Distributed VCS (Git)
- 3 workflow phổ biến quản lý mã nguồn: Git Flow, GitHub Flow, Trunk-Based Development (TBD) -> phổ biến nhất: GitHub Flow
- Nên tuân thủ quy tắc đặt tên nhánh: ngắn gọn, có ý nghĩa, độc lập
- nhiệm vụ, theo môi trường, có tiền tố phân loại (feat/, bugfix/,...)
- GitHub Workflow có 6 bước: tạo nhánh từ nhánh mặc định, thay đổi code, tạo Pull Request, xử lý phản hồi, Merge Pull Request, xóa nhánh
- Phân biệt và sử dụng được các câu lệnh git thông dụng: git restore, git stash, git revert, git reset, git pull, git merge, git rebase, git cherry-pick